

Connecting SantaCruz to Adam

What has been built, the one decision everything else follows from, and the four questions that need answering before this touches a real booking.

Test data only

Adam is in a testing and training phase. Every name, address, phone number and price on this page and in the screens it describes is invented. No real customer data has been through this system.

CONTEXT

What Adam is, and what changes

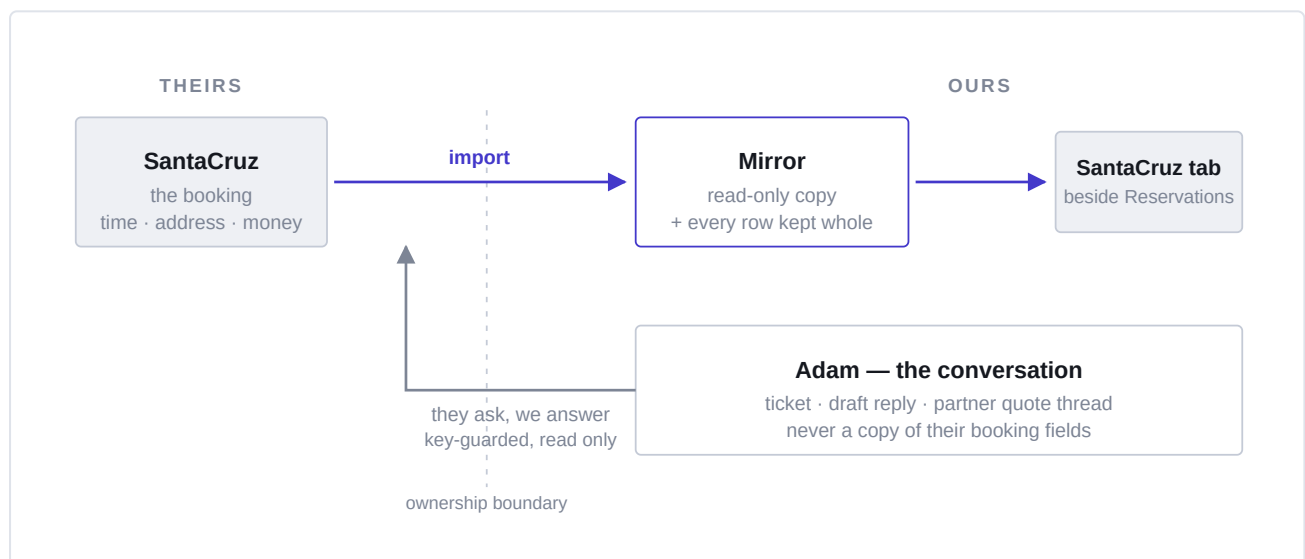
Support Desk turns email into tickets. Mail arrives at a Gmail inbox, becomes a ticket, gets sorted, and an agent called Adam drafts a reply that a person reviews before anything is sent. It runs as one service with a Postgres database behind it.

Its Operations screen — rosters, partners, reservations, dispatch — currently runs on invented data. The company is moving to **SantaCruz**, GroundWidgets' reservation, dispatch and accounting system. That makes SantaCruz the system of record for real bookings, and raises the question this document answers: how do the two systems share a booking without ending up with two versions of it?

SantaCruz owns the booking. Adam owns the conversation.

Everything below follows from this. SantaCruz is the truth for anything a driver or an invoice depends on — pickup time, addresses, who is travelling, the money. Adam owns what SantaCruz has no place for: the email ticket, the drafted reply, the quote thread with partners.

The practical consequence is that **Adam never writes a booking back to SantaCruz**. It holds a copy for display, and when the two disagree, theirs wins — because ours is a copy of their record rather than a second opinion about it. Two systems both able to edit the same pickup time is how a customer ends up with two cars at one kerb, and across a company boundary neither side can see or undo the other's change.



Bookings travel one way. The only thing crossing back is an answer to a question SantaCruz asked, about the conversation rather than about the booking — so there is never a second copy of a pickup time in circulation.

Their column names appear nowhere in the code

SantaCruz's real schema is not known yet and will change. Rather than guess it, the import is shape-agnostic: it accepts any set of columns, keeps every row exactly as it arrived, and a mapping edited on screen says which of their columns feeds which of our fields.

When GroundWidgets supply the real names, someone types them into a table. No migration, no deploy. A column nothing is mapped to is not discarded — it is stored with the row and reported after each import, so a field that turns out to matter in six months is already there.

The mapping, as configured for a test export				WORKING
ADAM'S FIELD	THEIR COLUMN	REQUIRED	READ AS	
externalId	RES_ID	yes	text — the idempotency key	
reference	RES_NO	—	text	
pickupAt	PU_DATETIME	yes	timestamp, in a configured zone	
pickupAddress	PU_ADDR	yes	text	
dropoffAddress	DO_ADDR	—	text	
passengerCount	PAX_CNT	—	whole number, ≥ 0	
luggageCount	BAG_CNT	—	whole number — blank stays blank	
priceCents	TOTAL	—	money, stored in whole cents	
Eighteen fields are offered in total. A mapping pointing at a field Adam does not have is refused rather than saved — a mapping that looks configured and quietly does nothing is worse than none.				

THE RULE

A row that cannot be read is refused, never approximated

The project's founding rule is that nothing is invented for a customer. At one email a time that means the agent asks instead of assuming. At table scale it means a bad value stops the row rather than becoming a plausible-looking default on every booking.

A refused row is kept whole and listed with its reason, so nothing has to be re-exported. These four are verbatim from a live run of a seven-row test export.

Live run — 3 rows in, 4 refused		REFUSED
ROW 4	pickupAt: "next Tuesday" is not a date and time we can read. Expected something like 2026-03-14 15:30.	
ROW 5	pickupAddress is required and "PU_ADDR" was empty	
ROW 6	priceCents: "about two hundred" is not an amount we can bill for priceCents	
ROW 7	pickupAt: "2026-11-01 01:30" happens twice in America/New_York on the day the clocks go back, so which one is meant is not in the data. The one worth arguing about. 01:30 on 1 November occurs twice in New York, and which occurrence is meant is genuinely absent from the file. Most importers pick one silently. An hour's error on an airport run misses the flight.	

A related case that passes rather than fails: an empty bag count is stored as *unknown* and displayed as “not stated”, never as zero. A driver told “3 passengers, 0 bags” brings a sedan and finds a boot's worth of luggage — that exact fault has already reached a driver once in this system, from a different code path.

Two directions, two secrets, one of them read-only

Inbound is a staff-facing API behind a login, with every write requiring an admin.

Outbound is a single endpoint SantaCruz calls with a shared secret and no login, mounted on a separate path so that a route added later cannot inherit the wrong protection by accident.

What SantaCruz can call

BUILT

```
GET /api/external/santacruz/bookings/:reference    x-api-key: <shared secret>
```

Returns whether the desk knows the booking, when it first and last saw it, and the linked ticket if there is one. Deliberately not their own booking fields: handing those back would create the second copy this design exists to avoid.

Two separate secrets — one for calling them, one for them calling us — so that leaking one does not hand over the other. Both live in the deployment's environment variables. The settings screen reports whether each is set and has no field to type one into; comparison is constant-time, and an unset secret refuses everybody rather than admitting everybody.

What is deliberately missing

CAPABILITY	STATE	WHY
Live API pull	BLOCKED	GroundWidgets have not published an integration surface. Import is by file until they do.
Webhooks	BLOCKED	Unknown whether SantaCruz can push. If not, polling cadence becomes a design question.
Linking a booking to a ticket	NOT STARTED	No reliable key exists yet. Guessing from a name or address would be exactly the invention this design refuses. See question 1.
Learning about cancellations	NOT STARTED	A booking deleted on their side stays in the mirror indefinitely. A real gap. See question 2.
Vehicle class translation	OPEN	Their words are stored as written and not mapped onto our sedan/SUV/van/sprinter rules.
Writing to SantaCruz	BY DESIGN	Not a gap. Follows from the ownership decision above.

Where input would change what gets built

1 What links their booking to our ticket?

A customer emails; Adam handles the conversation; the same job exists in SantaCruz. Nothing currently connects the two. Booker email is unreliable — assistants book for clients, and the same address raises many bookings. Does SantaCruz carry a field we could put our ticket number in, or one it would echo back?

2 How do we learn a booking was cancelled or deleted?

A file export tells us what exists, never what stopped existing. A cancelled job would sit in the mirror looking live. Options: a status column we treat as authoritative, a full-replace import for a known window, or a deletions feed. This needs deciding before the mirror is trusted operationally.

3 Is their reservation id stable across edits?

The whole import is idempotent on it — re-running a file corrects rather than duplicates. If SantaCruz reissues an id when a booking is amended, that assumption breaks and every amendment becomes a duplicate booking.

4 Does SantaCruz store local time or UTC?

We currently read bare timestamps as wall-clock in a zone chosen on the settings screen, and refuse the two ambiguous hours a year outright. If they store UTC, or tag the zone, that refusal is unnecessary and should be relaxed rather than kept as a safeguard nobody needs.

5 Is refusing the right default at their volume?

On a seven-row test, four refusals are readable. On a nightly file of several hundred, thirty refusals need a workflow — who reviews them, and by when — or the rule quietly becomes “bookings go missing”. What volume should this be designed for?

The questions worth asking before signing

SantaCruz is not yet in use, which means integration capability is still a purchasing question rather than a support ticket. In rough order of how much they change the build: is there a read API for reservations, drivers, vehicles and affiliates, and is it documented? Can it push a webhook on create and change, or must we poll? Is read-only database access available, and to what — a replica, a reporting view, the live database? Failing all of those, can it export on a schedule, to where, in what format? Is there a write path at all? What does any of it cost, and is it a licence, a per-call charge or a services engagement? And is there a sandbox, so this can be finished without touching live bookings?

STATE

Where it stands today

751 backend tests passing	69 frontend tests passing	5 new tables, nothing altered	0 real bookings touched
Deployed and exercised end to end against a test export. The existing Operations screens are untouched — the mirror is a separate tab beside them, so nobody has to guess which system a job came from while both are in use. What remains is a real export and real column names, and both come from GroundWidgets.			

Support Desk · SantaCruz integration · prepared for development team review.

All data shown is invented. Adam has not been run against real customer records.